

Les fragments dans Thymeleaf

1. Qu'est-ce qu'un fragment ?

Un **fragment** est une partie d'un template. Thymeleaf permet d'importer des fragments d'un template dans un autre : c'est le mécanisme de base pour factoriser les éléments répétés d'un site (menu, en-tête, pied de page, boîtes de dialogue...) et pour construire des mises en page communes. Il facilite beaucoup la conception de l'interface d'un site Web, car on écrit chaque élément une seule fois.

Pour importer un fragment, il faut décrire **sa position** : dans quel template il se trouve, et comment le sélectionner dans ce template. Plusieurs méthodes de sélection existent :

- par **nom de fragment** : les éléments portant l'attribut `th:fragment="monFragment"` ;
- par **nom de balise** : tous les éléments d'un type donné (title, link, footer...) ;
- par **identifiant** : l'élément dont l'attribut `id` a une valeur donnée ;
- par **classe CSS** : tous les éléments portant une classe donnée.

2. Syntaxe des expressions de fragment

Une position de fragment s'écrit avec une **expression de fragment** :

```
~{chemin/du/template :: sélecteur}
```

Le chemin est celui du template **sans extension** et **sans / initial**, relatif au dossier des templates (`src/main/resources/templates/`). Par exemple, `~{fragments/my-template :: my-fragment1}` désigne le fichier `templates/fragments/my-template.html`.

Expression	Sélection
<code>~{fragments/tp1 :: nomFragment}</code>	Fragment déclaré par <code>th:fragment="nomFragment"</code>
<code>~{fragments/tp1 :: footer}</code>	Toutes les balises <code><footer></code> (nom de balise)
<code>~{fragments/tp1 :: #my-id}</code>	Élément dont l'id est <code>my-id</code>
<code>~{fragments/tp1 :: .my-class}</code>	Éléments portant la classe <code>my-class</code>
<code>~{fragments/tp1 :: p.my-class}</code>	Balises <code><p></code> portant la classe <code>my-class</code>
<code>~{fragments/tp1 :: head/script}</code>	Balises <code><script></code> enfants directs de <code><head></code>
<code>~{fragments/tp1}</code>	Le template entier
<code>~{:: sélecteur}</code> ou <code>~{this :: sélecteur}</code>	Fragment du template courant

Remarque. Les sélecteurs de balises, d'identifiants et de classes (*markup selectors*) s'inspirent de XPath et des sélecteurs CSS. Seuls les plus courants sont présentés ici ; la documentation officielle donne la liste complète.

3. th:insert, th:replace et th:include

Un template de fragments

Le fichier fragments/my-template.html contient plusieurs éléments que d'autres templates peuvent importer :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title>Mon template</title>
  <link rel="stylesheet" th:href="@{/main.css}" />
  <link rel="stylesheet" th:href="@{/secondary.css}" />
  <script th:src="@{/main.js}"></script>
  <script th:src="@{/secondary.js}"></script>
</head>
<body>
  <!-- Script dans Le body -->
  <script th:src="@{/script-inbody.js}"></script>

  <ul th:fragment="my-fragment1">
    <li><a th:href="@{/}">Accueil</a></li>
    <li><a th:href="@{/products}">Produits</a></li>
    <li><a th:href="@{/about}">À propos</a></li>
  </ul>

  <ul th:fragment="my-fragment2">
    <li><a th:href="@{/admin/products}">Gestion des produits</a></li>
    <li><a th:href="@{/admin/orders}">Gestion des commandes</a></li>
  </ul>

  <div id="my-id1">
    Élément DIV avec id = 'my-id1'
  </div>

  <p class="my-class">Élément P avec la classe my-class</p>
  <p>Élément P sans classe</p>
  <p class="my-class">Élément P avec la classe my-class</p>
</body>
</html>
```

Un template qui les utilise

my-page.html importe plusieurs fragments de my-template.html :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title th:replace=~{fragments/my-template :: title}>Titre</title>
  <th:block th:insert=~{fragments/my-template :: link}></th:block>
  <th:block th:insert=~{fragments/my-template :: head/script}></th:block>
</head>
<body>
  <h1>Ma page</h1>
  <p>Contenu de ma page</p>

  <div th:insert=~{fragments/my-template :: my-fragment1}></div>
  <div th:insert=~{fragments/my-template :: my-fragment2}></div>
  <div th:insert=~{fragments/my-template :: #my-id1}></div>
  <div th:insert=~{fragments/my-template :: p.my-class}></div>
</body>
</html>
```

Résultat obtenu :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>Mon template</title>
  <link rel="stylesheet" href="/main.css" />
  <link rel="stylesheet" href="/secondary.css" />
  <script src="/main.js"></script>
  <script src="/secondary.js"></script>
</head>
<body>
  <h1>Ma page</h1>
  <p>Contenu de ma page</p>

  <div>
    <ul>
      <li><a href="/">Accueil</a></li>
      <li><a href="/products">Produits</a></li>
      <li><a href="/about">À propos</a></li>
    </ul>
  </div>
  <div>
    <ul>
      <li><a href="/admin/products">Gestion des produits</a></li>
      <li><a href="/admin/orders">Gestion des commandes</a></li>
    </ul>
  </div>
```

```

    </ul>
  </div>
  <div>
    <div id="my-id1">
      Élément DIV avec id = 'my-id1'
    </div>
  </div>
  <div>
    <p class="my-class">Élément P avec la classe my-class</p>
    <p class="my-class">Élément P avec la classe my-class</p>
  </div>
</body>
</html>

```

Remarques :

- Le sélecteur `title` remplace le titre par celui du template importé ; `link` importe **toutes** les balises `<link>` ; `head/script` importe les `<script>` situés directement dans le `<head>` (et non celui du `<body>`).
- `<th:block>` est un conteneur virtuel : il disparaît du résultat, ce qui permet d'insérer plusieurs éléments sans balise HTML supplémentaire.
- Le sélecteur `p.my-class` importe **deux** paragraphes : un sélecteur peut désigner plusieurs éléments.

Les trois attributs

Attribut	Effet
<code>th:insert</code>	Insère le fragment à l' intérieur de la balise cible (le fragment devient son enfant)
<code>th:replace</code>	Remplace la balise cible par le fragment
<code>th:include</code>	Insère le contenu du fragment (sans sa balise englobante) dans la balise cible — déprécié

Prenons le fragment suivant, dans `path/mytemplate.html` :

```

<footer th:fragment="copyright">
  &copy; Mon site
</footer>

```

Template cible :

```

<body>
  <!-- th:insert -->
  <div th:insert="~{path/mytemplate :: copyright}"></div>

  <!-- th:replace -->
  <div th:replace="~{path/mytemplate :: copyright}"></div>

  <!-- th:include (déprécié) -->

```

```
<div th:include=~{path/mytemplate :: copyright}></div>
</body>
```

Résultat :

```
<body>
  <!-- th:insert -->
  <div>
    <footer>
      &copy; Mon site
    </footer>
  </div>

  <!-- th:replace -->
  <footer>
    &copy; Mon site
  </footer>

  <!-- th:include -->
  <div>
    &copy; Mon site
  </div>
</body>
```

th:include est déprécié. Cet attribut date de Thymeleaf 2 ; il est déprécié depuis Thymeleaf 3 et ne doit plus être utilisé dans du nouveau code. On le remplace par `th:insert` ou `th:replace`, et, si l'on veut obtenir seulement le contenu d'un fragment, on le place dans un `<th:block>` à l'intérieur du fragment. **En pratique, on utilise le plus souvent th:replace** : le résultat est plus propre (pas de balise intermédiaire) et la maquette reste affichable telle quelle dans un navigateur.

Deux compléments :

- Le fragment inséré s'exécute dans le contexte du template appelant : il a accès aux **variables** de ce template (variables du modèle, variables locales).
- Le fragment peut aussi se trouver dans le même fichier : `th:replace=~{:: copyright}` (ou `~{this :: copyright}`).

4. Fragments paramétrés

Un fragment peut recevoir des paramètres, comme une fonction.

Paramètres explicites

Lorsqu'on énumère les paramètres dans la déclaration, ils deviennent **obligatoires** :

```
<div th:fragment="person (firstName, lastName)" class="box">
  <p>Prénom : <span th:text="${firstName}"></span></p>
  <p>Nom : <span th:text="${lastName}"></span></p>
```

```
<p>Nom complet : <span th:text="|${firstName} ${lastName}|"></span></p>
</div>
```

Appels possibles (par position, ou par nom, ce qui permet de changer l'ordre) :

```
<div th:replace=~{fragments/my-template2 :: person('Ada',
'Lovelace')}></div>

<div th:replace=~{fragments/my-template2 ::
    person(firstName='Ada', lastName='Lovelace')}></div>

<div th:replace=~{fragments/my-template2 :: person(${u.firstName},
${u.lastName})}></div>

<div th:replace=~{fragments/my-template2 ::
    person(firstName=${u.firstName},
lastName=${u.lastName})}></div>
```

Si le nombre de paramètres de l'appel ne correspond pas à la déclaration, Thymeleaf lève une exception.

Paramètres implicites

On peut aussi déclarer un fragment **sans liste de paramètres** et lui passer des valeurs à l'appel : elles sont alors disponibles comme des variables locales du fragment. Ces paramètres sont **facultatifs** : si on ne les fournit pas, les variables sont simplement absentes (null).

```
<!-- Fragment sans paramètres déclarés -->
<div th:fragment="greeting" class="box">
    <p>Bonjour
        <span th:text="${title}"></span>
        <span th:text="${name} ?: 'inconnu'"></span>
    </p>
</div>

<!-- Appel avec des valeurs -->
<div th:replace=~{fragments/my-template2 :: greeting(title='M.',
name='Tom')}></div>

<!-- Appel sans valeur : title et name sont absents -->
<div th:replace=~{fragments/my-template2 :: greeting}></div>
```

Dans le second appel, title est vide et name prend la valeur par défaut inconnu grâce à l'opérateur Elvis (voir le document sur les opérateurs conditionnel et Elvis).

5. Passer du balisage comme paramètre : les mises en page

Un paramètre de fragment peut lui-même être une **expression de fragment** (`~{...}`). Cette technique est à la base des **mises en page** (*layouts*) : un gabarit commun définit la structure de la page (menu, pied de page), et chaque page lui fournit son titre et son contenu.

Le gabarit, `templates/layout.html` :

```
<!DOCTYPE html>
<html th:fragment="layout (title, content)"
xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8" />
  <title th:replace="${title}">Titre du gabarit</title>
  <link rel="stylesheet" th:href="@{/css/style.css}" />
</head>
<body>
  <div th:replace="~{fragments/_menu :: menu}"></div>

  <main th:replace="${content}">
    <p>Contenu du gabarit</p>
  </main>

  <footer>&copy; Mon site</footer>
</body>
</html>
```

Une page qui utilise ce gabarit :

```
<!DOCTYPE html>
<html th:replace="~{layout :: layout(~{::title}, ~{::main})}"
xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Liste des produits</title>
</head>
<body>
  <main>
    <h1>Produits</h1>
    <p>Contenu propre à cette page.</p>
  </main>
</body>
</html>
```

Ici, `~{::title}` et `~{::main}` désignent le `<title>` et le `<main>` **de la page courante** ; ils sont transmis au gabarit, qui les place là où il utilise `${title}` et `${content}`. Chaque page ne contient donc que ce qui lui est propre.

Pour aller plus loin. Il existe aussi une bibliothèque tierce, *Thymeleaf Layout Dialect*, qui propose une approche par héritage de gabarits (`layout:decorate`).

Elle est indépendante de Thymeleaf et n'est pas nécessaire pour les besoins courants.

Expressions de fragment dans une condition

Une expression de fragment peut être utilisée dans un test ; `~{ }` désigne le **fragment vide** (rien n'est inséré) :

```
<div th:insert="${user != null} ? ~{fragments/_menu :: userMenu} :
~{fragments/_menu :: guestMenu}"></div>

<div th:insert="${showBanner} ? ~{fragments/_banner :: banner} : ~{ }"></div>
```

6. th:assert

L'attribut `th:assert` évalue une ou plusieurs expressions séparées par des virgules. Si **toutes** sont vraies, le traitement continue ; si l'une est fausse, une exception est levée (le rendu de la page échoue). Il est utile pour **vérifier les paramètres** d'un fragment :

```
<div th:fragment="employee (firstName, lastName)" class="box"
    th:assert="${!#strings.isEmpty(firstName)},
               ${!#strings.isEmpty(lastName)}">
    <p>Prénom : <span th:text="${firstName}"></span></p>
    <p>Nom : <span th:text="${lastName}"></span></p>
    <p>Nom complet : <span th:text="${firstName} ${lastName}"></span></p>
</div>
```

Appeler `employee('', 'Dupont')` lève une exception, puisque le prénom est vide. `th:assert` détecte une erreur de programmation ; ce n'est pas un outil de validation de saisie utilisateur (pour cela, voir Bean Validation).

7. Retourner un fragment depuis un contrôleur

Un contrôleur Spring peut retourner **uniquement un fragment** comme vue, avec la syntaxe template `:: sélecteur`. C'est très pratique pour rafraîchir une partie de la page sans la recharger, notamment avec **htmx** ou des appels `fetch` :

```
@GetMapping("/persons/rows")
public String personRows(Model model) {
    model.addAttribute("persons", service.findAll());
    return "personList :: rows"; // template personList.html, fragment «
rows »
}

<!-- personList.html -->
<tbody th:fragment="rows">
    <tr th:each="p : ${persons}">
        <td th:text="${p.firstName}">Prénom</td>
```



```
<td th:text="{p.lastName}">Nom</td>
</tr>
</tbody>
```

La réponse HTTP ne contient alors que le `<tbody>` : un petit script ou htmx peut le substituer dans la page déjà affichée. On limite ainsi la quantité de données échangées entre le client et le serveur, ce qui est pertinent dans une architecture répartie.

8. Sécurité et bonnes pratiques

Ne construisez jamais un nom de template ou de fragment à partir d'une saisie utilisateur. Une expression comme `th:insert=~{{param.page}} :: content}`, ou un contrôleur qui retourne `"users/" + paramUtilisateur` comme nom de vue, permet à un attaquant d'injecter une expression Thymeleaf et d'exécuter du code côté serveur (*Server-Side Template Injection*). Thymeleaf 3.0.12 puis 3.1 ont ajouté des restrictions, mais la règle reste : les noms de templates et de fragments sont **toujours des constantes**, ou choisis dans une liste blanche connue.

Autres bonnes pratiques :

- Rangez les fragments dans un dossier dédié (`templates/fragments/`), et nommez-les de façon explicite.
- Préférez `th:replace` à `th:insert` pour un balisage plus propre.
- Documentez les paramètres d'un fragment et vérifiez-les avec `th:assert` lorsqu'ils sont obligatoires.
- Utilisez `th:text` et non `th:utext` dans les fragments : ils sont réutilisés dans de nombreux contextes.
- Évitez les fragments imbriqués trop profondément : le chemin d'un élément devient difficile à suivre.

Résumé

- Un fragment est une partie de template, désignée par `~{template :: sélecteur}` (nom de fragment, balise, `#id`, `.classe`, chemin de balises).
- `th:insert` insère le fragment dans la balise cible, `th:replace` remplace la balise, `th:include` est déprécié.
- Les fragments acceptent des paramètres explicites (obligatoires) ou implicites (facultatifs), et peuvent recevoir d'autres fragments en paramètre : c'est la base des mises en page.
- `th:assert` vérifie des conditions et lève une exception si l'une est fausse.
- Un contrôleur peut retourner un fragment seul (`"template :: fragment"`), utile avec htmx.
- Les noms de templates et de fragments ne doivent jamais dépendre d'une saisie utilisateur.

Exercices proposés

1. Extraire l'en-tête et le pied de page des trois pages du TP Spring Boot dans `fragments/_layout.html`, et les importer avec `th:replace`.
2. Écrire un fragment `alert(type, message)` affichant une boîte colorée selon le type (`success`, `warning`, `danger`), et l'appeler avec des paramètres nommés.
3. Reprendre l'exemple de gabarit `layout` (`title`, `content`) et ajouter un troisième paramètre `extraScripts`, facultatif, pour des scripts propres à certaines pages.
4. Écrire un fragment `pagination(page, totalPages)` protégé par `th:assert` (la page doit être comprise entre 1 et `totalPages`).
5. Retourner le fragment `rows` depuis un contrôleur et le rafraîchir avec `htmx` (`hx-get`, `hx-target`) toutes les cinq secondes.